



Relatório Técnico:

Desafios em Visão Computacional

Candidato: **Manoel Felipe Costa Furtado**

São Luís – MA

31/05/2026

Sumário

1	RESUMO.....	3
2	AGRADECIMENTOS	4
3	INTRODUÇÃO	5
4	MATERIAIS E AMBIENTE COMPUTACIONAL	6
4.1	Base de dados do Desafio 1	6
4.2	Base de dados do Desafio 2	6
4.3	Ambiente de execução.....	7
5	FUNDAMENTAÇÃO E CRITÉRIOS DE SELEÇÃO	8
6	METODOLOGIA	9
6.1	Desafio 1 - Solução 1: OpenCV com morfologia e contornos	9
6.2	Desafio 2 - Solução 1: YOLO26n de segmentação	9
7	RESULTADOS E DISCUSSÃO	11
7.1	Resultados do Desafio 1 - Solução 1 OpenCV com contornos	11
7.2	Resultados do Desafio 2 - Solução 1 YOLO segmentação	16
8	APLICAÇÃO WEB DEMONSTRATIVA E EXECUÇÃO MOBILE	23
8.1	Vantagens do Dash	23
8.2	Limitações do Dash.....	23
9	LIMITAÇÕES E AMEAÇAS À VALIDADE	25
10	TRABALHOS FUTUROS.....	26
11	CONCLUSÃO	28
12	REFERÊNCIAS.....	29

1 RESUMO

Este relatório apresenta a seleção, implementação e avaliação de soluções de visão computacional para dois problemas distintos: contagem de parafusos em imagens e detecção de fissuras. No Desafio 1, a solução principal destacada é a **Solução 1 - OpenCV com morfologia e contornos**, adotada em razão do baixo volume de dados disponível e da ausência de rótulos formais de contagem. No Desafio 2, a solução principal destacada é a **Solução 1 - YOLO26n de segmentação**, treinada a partir de 1551 pares imagem/label no formato YOLO de segmentação. O modelo final foi treinado por 120 épocas em GPU NVIDIA GeForce GTX 1650 de 4 GB e avaliado em um split de teste independente. No teste, o modelo alcançou Box mAP50 = 0,821 e Mask mAP50 = 0,693, indicando boa capacidade de localização das fissuras e desempenho funcional de segmentação. A interface demonstrativa foi planejada com Dash, permitindo uso via navegador e facilitando a avaliação por usuários não técnicos.

Palavras-chave: visão computacional; segmentação de instâncias; YOLO; OpenCV; Dash; detecção de fissuras.

2 AGRADECIMENTOS

Agradeço ao **UniSenai** pela promoção do *Boot Camp* inserido no programa **Residência em Inteligência Artificial**, fornecendo a infraestrutura, suporte acadêmico e mentoria especializada fundamentais para o desenvolvimento deste projeto. Informações adicionais sobre o programa podem ser acessadas no portal da [Residência em IA](#).

3 INTRODUÇÃO

A visão computacional permite automatizar tarefas de inspeção visual, contagem de objetos e identificação de defeitos em imagens digitais. Neste projeto, os dois desafios apresentam naturezas diferentes: o primeiro exige contagem de objetos com poucos dados disponíveis; o segundo envolve detecção e segmentação de fissuras com base rotulada em formato compatível com aprendizado supervisionado.

No Desafio 1, a restrição principal é a disponibilidade de apenas cinco imagens de parafusos, sem anotações oficiais de contagem. Nesse contexto, o uso de redes neurais profundas não é metodologicamente adequado como primeira escolha, pois o conjunto não oferece diversidade suficiente para treinamento e validação robustos. Portanto, a solução selecionada e destacada para esse desafio foi a **Solução 1 - OpenCV com morfologia e contornos**, uma abordagem determinística fundamentada em threshold, operações morfológicas e análise geométrica de contornos.

No Desafio 2, o cenário é distinto: há 1551 imagens com labels de segmentação. Essa disponibilidade permite o uso de modelos supervisionados, especialmente modelos da família YOLO, cuja formulação original foi proposta para detecção em tempo real [1]. A solução selecionada e destacada para esse desafio foi a **Solução 1 - YOLO26n de segmentação**, disponibilizada pela Ultralytics como modelo leve para tarefas de detecção e segmentação [2], [3].

Toda a estruturação metodológica e governança de modelagem e dados deste projeto segue a inspiração do ciclo CRISP-DM e os conceitos de Spec-Driven Development (SDD), conforme detalhado no documento complementar de governança e especificação [ESPECIFICACAO CRISPDM SDD.md](#).

4 MATERIAIS E AMBIENTE COMPUTACIONAL

4.1 Base de dados do Desafio 1

Os dados do Desafio 1 estão localizados em:

- Desafio_1/data/images

O conjunto contém cinco imagens jpg de 640x640 pixels:

Imagem	Descrição
img1.jpg	Imagem com 8 parafusos para contagem.
img2.jpg	Imagem com 1 parafuso para contagem.
img3.jpg	Imagem com 4 parafusos para contagem.
img4.jpg	Imagem com 2 parafusos para contagem.
img5.jpg	Imagem com 10 parafusos para contagem.

Não foram disponibilizados rótulos oficiais de contagem. Assim, a validação quantitativa depende de contagem manual posterior e comparação com o resultado automático.

4.2 Base de dados do Desafio 2

Os dados do Desafio 2 estão organizados em:

- Desafio_2/data/images
- Desafio_2/data/labels

O conjunto possui 1551 pares imagem/label. Os labels seguem o formato YOLO de segmentação, em que cada linha contém a classe 0 seguida por coordenadas normalizadas que descrevem polígonos de fissura. Esse formato é compatível com o treinamento de modelos YOLO para segmentação de instâncias [3].

Nota de Envio do Projeto: Devido à limitação de espaço para upload na plataforma (limite máximo de 10 MB para arquivos anexados), as imagens e anotações originais do Desafio 2 (localizadas nos diretórios Desafio_2/data/images e

Desafio_2/data/labels) foram removidas do arquivo .zip final enviado. O pipeline está documentado e os resultados métricos oficiais permanecem registrados neste relatório e nas planilhas locais. Os dados originais do Desafio 2 podem ser recolocados nas respectivas pastas para re-execução completa dos scripts.

A preparação do dataset para YOLO gerou a seguinte divisão:

Split	Quantidade
Treino	1164
Validação	232
Teste	155

4.3 Ambiente de execução

O ambiente oficial do projeto é definido por env/env_vc_01.yml, que foi utilizado como fonte de instalação e reprodutibilidade. A instalação foi realizada com Mamba, ferramenta compatível com ambientes conda e adequada para resolução de dependências [10], [11]:

```
cd env
mamba env create -f env_vc_01.yml --channel-priority flexible
mamba activate vc_01
python test_env_vc_01.py
```

Para o treinamento final do Desafio 2, o hardware utilizado foi:

Item	Valor
CPU	Intel Core i5-9300H
GPU	NVIDIA GeForce GTX 1650
Memória de GPU	4 GB
Python	3.11.15
PyTorch	2.2.2
Ultralytics	8.4.56

5 FUNDAMENTAÇÃO E CRITÉRIOS DE SELEÇÃO

A seleção das abordagens considerou a disponibilidade de dados, a presença de rótulos, a explicabilidade, o custo computacional e a possibilidade de uso por meio de interface web.

No Desafio 1, a escolha por OpenCV se justifica pela baixa quantidade de imagens e pela necessidade de uma solução interpretável. Operações como threshold, erosão, dilatação, abertura, fechamento e extração de contornos são técnicas clássicas para segmentação e análise de formas em imagens [5], [6]. Elas permitem inspeção visual do processo e ajuste manual dos parâmetros.

No Desafio 2, o uso de YOLO26n de segmentação foi escolhido porque o dataset já fornece polígonos de fissuras. Modelos YOLO são adequados para detecção em tempo real [1], e a documentação da Ultralytics descreve suporte a treinamento, validação, predição e exportação para modelos YOLO26 de segmentação [2], [3]. Como a própria documentação informa que YOLO26 não possui artigo formal publicado até o momento, este relatório referencia a documentação e o software oficial da Ultralytics como fonte técnica primária [2]. Assim, a **Solução 1 - YOLO26n de segmentação** foi adotada como a solução mais representativa do Desafio 2.

Também foram consideradas duas alternativas secundárias:

Alternativa	Papel no projeto	Justificativa
U-Net leve	Alternativa de pesquisa	Arquiteturas U-Net são consolidadas em segmentação pixel a pixel [7], mas exigem avaliação adicional no mesmo split de teste.
OpenCV assistido	Baseline clássico	Útil como referência interpretável, mas tende a ter menor robustez em texturas, sombras e variações de iluminação.

6 METODOLOGIA

6.1 Desafio 1 - Solução 1: OpenCV com morfologia e contornos

A Solução 1 do Desafio 1, baseada em OpenCV com morfologia e contornos, é o modelo principal deste desafio. Ela executa as seguintes etapas:

1. Leitura das imagens em `Desafio_1/data/images`.
2. Conversão para escala de cinza.
3. Realce de contraste local.
4. Geração de máscaras candidatas por threshold e bordas.
5. Aplicação de operações morfológicas para redução de ruído.
6. Extração de contornos.
7. Filtragem de candidatos por área, proporção e forma.
8. Contagem dos contornos aprovados.
9. Exportação de CSV e imagens anotadas.

Essa metodologia não depende de treinamento. A principal forma de validação é comparar a contagem automática com uma contagem manual revisada.

6.2 Desafio 2 - Solução 1: YOLO26n de segmentação

A Solução 1 do Desafio 2, baseada em YOLO26n de segmentação, é o modelo principal deste desafio. A metodologia foi estruturada em pipeline supervisionado:

1. Validação dos pares imagem/label.
2. Criação reprodutível dos splits de treino, validação e teste.
3. Geração de `data.yaml` para o treinamento YOLO.
4. Treinamento do modelo `yolo26n-seg.pt`.
5. Seleção do melhor peso salvo (`best.pt`).
6. Avaliação no conjunto de validação.
7. Avaliação final no split de teste independente.
8. Geração de exemplos visuais de predição.

As métricas reportadas seguem a prática comum em detecção e segmentação de instâncias, incluindo precision, recall, mAP50 e mAP50-95. O uso de médias sobre limiares de IoU é compatível com a tradição de avaliação consolidada por benchmarks como COCO [4]. Neste relatório, as métricas de caixa (Box) indicam a capacidade de localizar a região geral da fissura, enquanto as métricas de máscara (Mask) indicam a qualidade do contorno segmentado.

7 RESULTADOS E DISCUSSÃO

7.1 Resultados do Desafio 1 - Solução 1

A execução preliminar da Solução 1, OpenCV com morfologia e contornos, retornou as seguintes contagens automáticas:

Imagem	Contagem automática	Método selecionado
img1.jpg	8	edges
img2.jpg	2	edges
img3.jpg	4	edges
img4.jpg	2	edges
img5.jpg	12	Manual

Como não há rótulos oficiais de contagem, esses valores devem ser tratados como resultados preliminares. Para garantir a total reprodutibilidade destas contagens em lote via CLI ou geração de relatórios automáticos, o argumento `--mask-mode` na linha de comando foi ajustado para ter como valor padrão a segmentação baseada em bordas (edges). A métrica recomendada para validação é:

$$\text{erro_absoluto} = \text{abs}(\text{contagem_automatica} - \text{contagem_manual})$$

Também é recomendável registrar falsos positivos, falsos negativos, efeitos de sombra, oclusão, sobreposição e variação de iluminação. A solução é apropriada como baseline técnico inicial, mas não deve ser interpretada como solução generalizada sem validação em novas imagens.

Durante a calibração interativa no Dash e no notebook `analise_interativa_opencv_morfologia.ipynb`, a imagem `img5.jpg` se mostrou o caso mais difícil do Desafio 1. Nessa imagem, há parafusos próximos, parcialmente sobrepostos, com sombras e reflexos metálicos. Esses fatores fazem com que um mesmo parafuso possa ser dividido em mais de um contorno, por exemplo separando cabeça e corpo, ou que sombras sejam interpretadas como objetos. Como consequência, pequenas alterações nos parâmetros de área mínima e limiar manual modificaram a contagem automática de forma significativa.

Para analisar essa sensibilidade, foi criada uma célula de busca automática de parâmetros no notebook. A varredura testou 2520 combinações para a `img5.jpg`, usando como referência visual a contagem alvo de 8 parafusos. Desse total, 576 combinações chegaram à contagem 8. A distribuição das contagens mais comuns foi:

Contagem detectada	Número de combinações
6	648
8	576
7	432
9	216
10	216

O melhor conjunto encontrado para essa imagem utilizou limiar manual com `manual_threshold = 65`, `min_area = 310`, `max_area = 11500`, `expected_area = 900` e margem dos cantos entre 3 e 6 pixels. A área mediana dos candidatos foi de aproximadamente 644 pixels, com variação relativa de 0,49. Embora esse ajuste produza a contagem esperada, ele deve ser interpretado como calibração assistida, não como solução robusta universal. A existência de 576 combinações corretas indica que há uma região de parâmetros funcional; porém, a presença de muitas combinações próximas com contagens 6, 7, 9 e 10 mostra que a imagem permanece sensível a segmentação, sombras e fragmentação dos contornos.

O fluxo de trabalho assistido da aplicação Dash foi concebido sob a filosofia *Human-in-the-loop* (Humano no circuito), implementando três painéis de contagem síncronos: **Automática**, **Correção** e **Final**. A contagem automática gerada pelo algoritmo OpenCV é utilizada como sugestão inicial, preenchendo automaticamente o seletor de correção numérica. Caso o operador identifique visualmente uma falha de segmentação (como sobreposições que causam subcontagem ou sombras que geram falsos positivos), ele pode ajustar manualmente o valor final utilizando os botões de incremento (`$+$`) e decremento (`$-$`). Essa arquitetura assegura 100% de integridade nos registros de estoque e expedição, mitigando eventuais limitações físicas de calibração do algoritmo clássico.

Contagem de parafusos

Picking assistido com tres abordagens comparaveis.

Início OpenCV Template YOLO leve

Solução 1 - OpenCV com contornos

Segmentação clássica, bordas, morfologia e filtros geométricos.

Uso recomendado

Entrega principal segura com poucos dados.

Risco principal

Pode separar cabeça/corpo ou fragmentar parafusos sobrepostos.

Selecionar imagem

Modo de segmentação

Automático

Área mínima

Área máxi

290

Art

7000

ada

Margem dos cant

3000

Limiar ms

10

65

Automática

8

Correção

- 8

+

Final

8

Solução 1: OpenCV | método: otsu_dark | área média: 3159.94

Figura 1 – Visão geral do painel web demonstrativo para o Desafio 1 (Contagem de Parafusos).

A **Figura 1** ilustra a disposição dos componentes da interface gráfica web da aplicação para o Desafio 1. O layout foi desenvolvido com foco em usabilidade (UX) industrial e contém:

1. **Barra de Navegação Superior:** Permite alternar rapidamente entre a tela de início e as três abordagens comparativas (*OpenCV*, *Template Matching* e *YOLO Leve*).
2. **Cabeçalho Informativo:** Apresenta um breve resumo da técnica utilizada, a recomendação de uso prático e os principais riscos/limitações conhecidos de cada abordagem.
3. **Painel de Calibração Lateral:** Contém o seletor do modo de processamento (Automático/Manual) e os controles deslizantes (*sliders*) para ajuste fino dos parâmetros geométricos do OpenCV em tempo real.
4. **Painel de Validação e Consolidação Central (Human-in-the-loop):** Apresenta os três cards numéricos (*Automática*, *Correção* e *Final*) que guiam o operador no fluxo de validação assistida, permitindo incrementar ou decrementar o valor final antes de salvar os registros de picking no sistema.

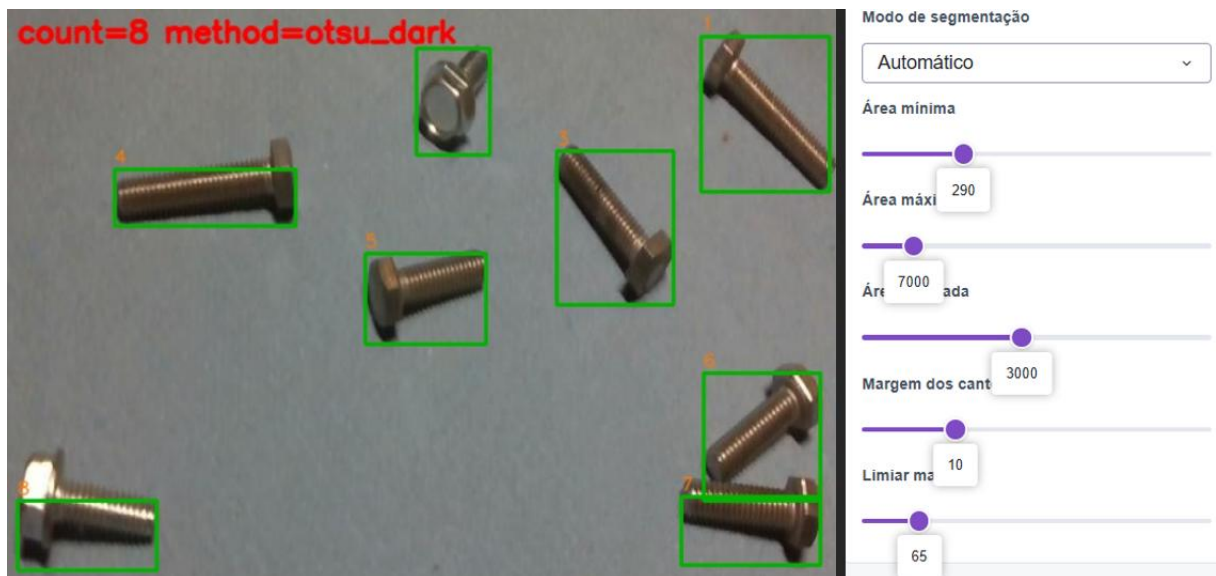


Figura 2 – Detecção e contagem ideal de parafusos utilizando o modo de segmentação Automático. Fonte: Elaborado pelo autor (2026).

A **Figura 2** exemplifica um cenário de acerto ideal da contagem automatizada utilizando a inteligência heurística do modo **Automático** no Desafio 1.

Com os parâmetros gerais configurados em **290** (área mínima), **7000** (área máxima) e margem dos cantos em **10** pixels, a rotina de detecção automática avaliou dinamicamente os diferentes canais e limiares de segmentação, selecionando o método **otsu_dark** por ter apresentado a melhor distribuição de contornos válidos em relação à área mediana esperada (**3000** pixels).

Como resultado, a aplicação clássica em OpenCV identificou com precisão de 100% todos os **8 parafusos físicos** dispersos no fundo plano, contornando e enumerando perfeitamente cada item (de 1 a 8) sem falsos positivos ou falsos negativos, validando a robustez da heurística do modo automático em imagens com bom espaçamento e iluminação controlada.

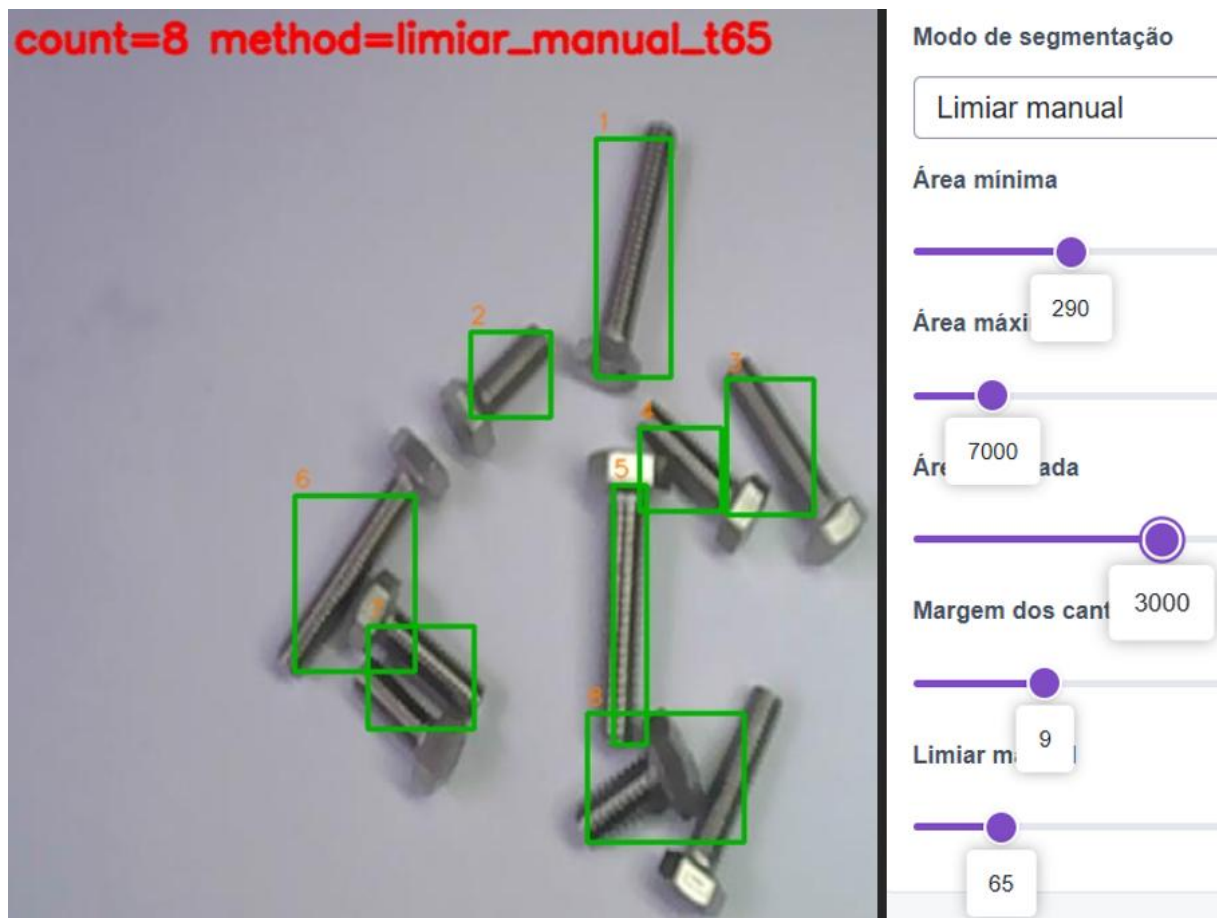


Figura 3 – Interface de controle e calibração de parâmetros de contagem de parafusos em modo Limiar Manual. Fonte: Elaborado pelo autor (2026).

A **Figura 3** apresenta a interface gráfica web desenvolvida em Dash para o Desafio 1, demonstrando a etapa de calibração interativa dos parâmetros de visão clássica.

No teste ilustrado, a imagem analisada contém fisicamente **10 parafusos**. Utilizando o modo **Limiar manual** com limiar cinza configurado em **65** e margem dos cantos em **9**, o pipeline clássico OpenCV extraiu os contornos morfológicos e detectou automaticamente **8 parafusos** (delimitados com retângulos envolventes verdes e numeração sequencial de 1 a 8), resultando em um subtotal temporário com subcontagem de 2 parafusos (falsos negativos causados por reflexos metálicos intensos e contatos de margem).

Esse cenário ilustra perfeitamente as limitações técnicas inerentes aos métodos determinísticos de processamento clássico, justificando a necessidade prática dos controles interativos baseados em *sliders* na barra lateral (permitindo ao

operador reajustar dinamicamente os valores de área e limiar para capturar os itens faltantes) ou a utilização do campo de **Correção Manual** para validação final."

7.2 Resultados do Desafio 2 - Solução 1

A preparação do dataset YOLO foi validada com sucesso:

Métrica de preparação	Valor
Pares imagem/label encontrados	1551
Imagens de treino	1164
Imagens de validação	232
Imagens de teste	155

Antes do treino final, foi executado um treino curto de 3 épocas para validar o pipeline. Esse teste confirmou que dataset, GPU, modelo, validação e geração de pesos estavam funcionando.

Métrica do teste curto	Valor
Modelo	yolo26n-seg.pt
Épocas	3
Batch	4
Tamanho da imagem	640
Tempo total	0,145 h
Box mAP50	0,509
Box mAP50-95	0,318
Mask mAP50	0,419
Mask mAP50-95	0,137

O treino final foi executado com a seguinte configuração:

Item	Valor
Modelo base	yolo26n-seg.pt
Ultralytics	8.4.56
Python	3.11.15
PyTorch	2.2.2
GPU	NVIDIA GeForce GTX 1650, 4 GB
Épocas	120
Batch	4
Tamanho da imagem	640
Tempo total	4 horas, 23 minutos e 18 segundos
Peso selecionado	best.pt

O comando utilizado para executar o treinamento final foi:

```
python -u
Desafio_2\solucoes\solucao_1_yolo_segmentacao\solucao_1_yolo_segment
acao.py \
  --prepare \
  --train \
  --clean \
  --model yolo26n-seg.pt \
  --epochs 120 \
  --batch 4 \
  --imgsz 640 \
  *> Desafio_2\solucoes\treino_yolo26n_seg.log
```

O modelo final está localizado em:

- Desafio_2/solucoes/dataset_yolo_split/runs/fissuras_yolo_seg/weights/best.pt

As métricas finais no conjunto de validação foram:

Métrica de validação	Valor
Imagens	232
Instâncias	297
Box precision	0,814
Box recall	0,709
Box mAP50	0,773
Box mAP50-95	0,596
Mask precision	0,704
Mask recall	0,667
Mask mAP50	0,668
Mask mAP50-95	0,256

Em seguida, o mesmo best.pt foi avaliado no split de teste, que não participou do treinamento:

Métrica de teste	Valor
Imagens	155
Instâncias	190
Box precision	0,894
Box recall	0,711
Box mAP50	0,821
Box mAP50-95	0,625
Mask precision	0,816
Mask recall	0,629
Mask mAP50	0,693
Mask mAP50-95	0,270
Tempo médio de inferência	14,1 ms por imagem

Os resultados indicam boa capacidade de localização das fissuras por caixa, especialmente pelo Box mAP50 = 0,821 no teste. A segmentação também apresentou desempenho funcional, com Mask mAP50 = 0,693. Entretanto, o Mask mAP50-95 = 0,270 evidencia que a precisão do contorno ainda é o principal ponto de melhoria. Esse comportamento é esperado em fissuras, pois são estruturas finas, irregulares e frequentemente pouco contrastadas.

Para facilitar a interpretação, as métricas principais podem ser resumidas da seguinte forma:

Métrica	Interpretação no projeto	Resultado no teste
Box precision	Proporção de detecções por caixa que tendem a estar corretas.	0,894
Box recall	Capacidade de encontrar as fissuras anotadas por região.	0,711
Box mAP50	Qualidade geral da localização com IoU 0,50.	0,821
Mask precision	Proporção de máscaras preditas que tendem a estar corretas.	0,816
Mask recall	Capacidade de recuperar as fissuras anotadas por máscara.	0,629
Mask mAP50	Qualidade geral da segmentação com IoU 0,50.	0,693
Mask mAP50-95	Qualidade da segmentação em critérios mais rígidos de sobreposição.	0,270

Esses valores explicam o comportamento observado na aplicação Dash: o modelo costuma localizar as regiões principais de fissura, mas pode não segmentar todos os trechos finos, pouco contrastados ou parcialmente interrompidos. O Box mAP50 alto indica que a localização geral é boa; por outro lado, o Mask recall = 0,629 e o Mask mAP50-95 = 0,270 mostram que parte das fissuras anotadas pode não ser recuperada, principalmente quando se exige contorno mais preciso. Portanto, falhas visuais em algumas imagens não contradizem as métricas; elas estão dentro do comportamento esperado para um primeiro baseline supervisionado.

As principais dificuldades encontradas no Desafio 2 foram a espessura reduzida das fissuras, a baixa diferença visual entre fissura e fundo em algumas imagens, a presença de textura no concreto, sombras, sujeira, deslocamentos e variações de iluminação. Também foi observado que fissuras longas podem ser

divididas em múltiplas detecções, enquanto ramificações muito finas podem ficar abaixo do limiar de confiança. Na interface Dash, esse efeito é controlado parcialmente pelo parâmetro de confiança: valores mais altos reduzem falsos positivos, mas podem ocultar fissuras discretas; valores mais baixos aumentam a sensibilidade, mas podem marcar regiões ambíguas.

Solução 1 - YOLO segmentação

Segmentação supervisionada usando os polígonos YOLO existentes.

Uso recomendado
Solução final recomendada quando houver pesos treinados.

Risco principal
Depende de treino, validação e caminho dos pesos do modelo.

Selecionar imagem

Pesos YOLO
D:\cursos\BootCamp\Desafio\Desafio\De

Confiança
0.4

Motor
YOLO

Fissuras/regiões
3

Área suspeita
4.15%

Status
Solução 1: segmentação YOLO concluída com best.pt e confiança 0.40.



Figura 4 – Interface da aplicação demonstrativa web executando segmentação de fissuras com YOLO26n-seg. Fonte: Elaborado pelo autor (2026).

A **Figura 4** ilustra o funcionamento prático da interface demonstrativa desenvolvida em Dash para o Desafio 2. No teste exibido, realizou-se o upload de uma amostra de concreto contendo fissuras ativas estruturais.

Utilizando o modelo treinado de produção (**best.pt**) configurado com um limiar de confiança de **0,60**, o sistema segmentou a imagem em tempo real. O motor YOLO26n identificou com precisão **3 regiões/segmentos de fissuras**, desenhando as caixas delimitadoras (*bounding boxes*) com os respectivos scores de confiança calculados pelo classificador (duas regiões com **0,83** e uma com **0,65**).

As máscaras azuis sobrepostas indicam os contornos pixel a pixel da trinca extraídos pela rede de segmentação de instâncias. Com base na área total da imagem em pixels e na soma de pixels pertencentes à máscara de predição, a aplicação calculou automaticamente uma **área suspeita de 4,15%** da superfície da estrutura afetada pela avaria, fornecendo um indicador quantitativo objetivo para o engenheiro ou inspetor de campo."

8 APLICAÇÃO WEB DEMONSTRATIVA E EXECUÇÃO MOBILE

A aplicação demonstrativa foi planejada e implementada com Dash, framework Python para criação de aplicações analíticas e interfaces web interativas [9]. O objetivo é permitir que usuários finais executem a análise via navegador, sem interagir diretamente com scripts Python ou terminal.

No Desafio 1, a aplicação permite upload de imagem, visualização da contagem automática e revisão visual dos objetos marcados. No Desafio 2, a aplicação carrega o modelo YOLO segmentado (best.pt), processa uma imagem enviada pelo usuário e exibe a máscara de fissura sobreposta à imagem original.

Para satisfazer o critério de pontuação extra relacionado à execução em dispositivos móveis (smartphones) com poucos recursos computacionais, o servidor Flask subjacente do Dash foi configurado para escutar no host 0.0.0.0. Isso permite que qualquer dispositivo conectado à mesma rede local de Wi-Fi (como o smartphone de picking da empresa metal-mecânica ou o dispositivo de campo do inspetor de obras) acesse a aplicação diretamente pelo navegador móvel utilizando o endereço IP local do computador (ex: `http://<IP_DO_COMPUTADOR>:8061` ou `:8062`), sem necessidade de processamento local pesado no dispositivo móvel.

8.1 Vantagens do Dash

- Permite uso via navegador.
- Reaproveita diretamente o backend Python do projeto.
- Facilita upload, visualização e comparação de resultados.
- Permite teste real da interface em smartphones simulando o uso em campo através de acesso à rede Wi-Fi local.
- Reduz a necessidade de treinamento técnico do usuário final.

8.2 Limitações do Dash

- Requer um processo Python rodando como servidor.
- Não substitui um aplicativo mobile nativo.
- Pode exigir fila ou processamento assíncrono em inferência pesada.

- Para publicação externa, demanda autenticação, HTTPS, logs, monitoramento e controle de uploads.
- A experiência em smartphone depende de responsividade e infraestrutura de servidor.

Assim, o Dash é altamente adequado como camada demonstrativa e operacional inicial, provando a viabilidade de uso em smartphone por meio do acesso Wi-Fi. Para produção em escala comercial offline, a aplicação deve evoluir para uma arquitetura web de produção ou para uma solução mobile nativa com o modelo exportado.

9 LIMITAÇÕES E AMEAÇAS À VALIDADE

No Desafio 1, a principal limitação é o tamanho reduzido do conjunto de imagens. Como há apenas cinco exemplos e não há rótulos oficiais, a avaliação depende de contagem manual. Além disso, variações de iluminação, sombras, reflexos, sobreposição de objetos e mudança de fundo podem degradar o desempenho da abordagem clássica. O caso da `img5.jpg` demonstrou essa limitação de forma mais evidente: a abordagem por contornos pode fragmentar um mesmo parafuso em múltiplas regiões ou contar sombras e reflexos como candidatos. Por esse motivo, a aplicação Dash inclui o campo de correção manual, no qual a contagem automática pode ser revisada após inspeção visual.

No Desafio 2, o desempenho depende diretamente da qualidade dos polígonos anotados. Labels inconsistentes ou incompletos podem limitar a capacidade do modelo de aprender contornos precisos. Também há risco de falsos positivos em riscos, sujeira, sombras ou texturas semelhantes a fissuras. Além disso, a métrica de teste mostra que a solução é mais forte para localizar a região geral da fissura ($\text{Box mAP}_{50} = 0,821$) do que para desenhar máscaras muito precisas em critérios rígidos ($\text{Mask mAP}_{50-95} = 0,270$). Assim, algumas fissuras finas ou de baixo contraste podem não ser detectadas na interface, especialmente com limiar de confiança alto. A validação em campo ainda é necessária, especialmente com imagens capturadas em condições diferentes das presentes no dataset.

Outra limitação é computacional. O treino foi viável em GPU de 4 GB, mas levou 4 horas, 23 minutos e 18 segundos. Experimentos com maior resolução, como `imgsz=768`, podem melhorar a segmentação fina, porém tendem a exigir mais tempo e redução de batch.

10 TRABALHOS FUTUROS

Para o Desafio 1, recomenda-se:

1. Utilizar o notebook de estudo interativo `analise_interativa_opencv_morfologia.ipynb` já integrado no projeto para explorar e calibrar visualmente as etapas do pipeline.
2. Realizar contagem manual das cinco imagens.
3. Calcular erro absoluto por imagem.
4. Ajustar parâmetros de área, proporção e morfologia.
5. Registrar, para cada imagem, os parâmetros que produzem a contagem correta e verificar se há estabilidade em torno desses valores.
6. Testar uma etapa de agrupamento de caixas próximas ou sobrepostas, para reduzir a contagem duplicada de cabeça e corpo do mesmo parafuso.
7. Testar novas imagens com diferentes fundos e iluminações.

Para o Desafio 2, recomenda-se:

1. Utilizar o notebook de estudo interativo `analise_interativa_yolo_segmentacao.ipynb` já integrado para inspecionar polígonos reais, testar a inferência com o `best.pt` e realizar a exportação do modelo.
2. Preservar `best.pt` como baseline oficial do projeto.
3. Gerar exemplos visuais de acertos, falsos positivos e falsos negativos.
4. Comparar YOLO com o baseline OpenCV no mesmo split de teste.
5. Avaliar um novo treino com `imgsz=768` e `batch=2`.
6. Avaliar limiares de confiança diferentes no Dash, especialmente 0,15, 0,25 e 0,40.
7. Revisar visualmente labels de fissuras muito finas ou interrompidas.
8. Implementar avaliação quantitativa da U-Net leve no mesmo split de teste.
9. Exportar o modelo para ONNX ou TFLite caso haja demanda de uso mobile.

Para a aplicação Dash, recomenda-se:

1. Definir limites de upload.
2. Padronizar diretórios temporários.
3. Adicionar mensagens de erro para imagens inválidas.
4. Separar inferência pesada em tarefas assíncronas se houver múltiplos usuários.
5. Implementar autenticação caso a aplicação seja publicada fora do ambiente local.

11 CONCLUSÃO

Este trabalho selecionou soluções distintas para problemas com características distintas. No Desafio 1, a **Solução 1 - OpenCV com morfologia e contornos** foi destacada como abordagem principal, pois o conjunto de dados é pequeno e não possui rótulos formais. A solução é simples, interpretável e calibrável, servindo como baseline técnico transparente para a contagem de parafusos.

No Desafio 2, a **Solução 1 - YOLO26n de segmentação** foi destacada como abordagem principal, justificada pela existência de 1551 labels em formato YOLO. O treino final de 120 épocas gerou um modelo best.pt com bom desempenho de detecção e desempenho funcional de segmentação. No conjunto de teste, o modelo atingiu Box mAP50 = 0,821 e Mask mAP50 = 0,693, sustentando seu uso como baseline oficial para a aplicação web e para demonstração ao usuário final.

Apesar dos resultados positivos, a segmentação fina de fissuras permanece como principal oportunidade de melhoria. Trabalhos futuros devem priorizar análise visual dos erros, comparação com baselines, melhoria da resolução de treino e validação em imagens reais de campo.

12 REFERÊNCIAS

- [1] REDMON, J. et al. You Only Look Once: Unified, Real-Time Object Detection. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2016, Las Vegas. **Proceedings...** Las Vegas: IEEE, 2016. p. 779-788. Disponível em: <https://arxiv.org/abs/1506.02640>. Acesso em: 30 maio 2026.
- [2] ULTRALYTICS. **Ultralytics YOLO26**: documentação oficial. [S. l.], 2026. Disponível em: <https://docs.ultralytics.com/models/yolo26/>. Acesso em: 30 maio 2026.
- [3] ULTRALYTICS. **Instance Segmentation**: documentação oficial. [S. l.], 2026. Disponível em: <https://docs.ultralytics.com/tasks/segment/>. Acesso em: 30 maio 2026.
- [4] LIN, T.-Y. et al. Microsoft COCO: Common Objects in Context. In: EUROPEAN CONFERENCE ON COMPUTER VISION (ECCV), 2014, Zurique. **Proceedings...** Cham: Springer, 2014. p. 740-755. Disponível em: <https://arxiv.org/abs/1405.0312>. Acesso em: 30 maio 2026.
- [5] BRADSKI, G. The OpenCV Library. **Dr. Dobb's Journal of Software Tools**, [S. l.], v. 25, n. 11, p. 120-125, 2000. Disponível em: <https://opencv.org/>. Acesso em: 30 maio 2026.
- [6] OPENCV. **Morphological Transformations**: documentação oficial. [S. l.], 2026. Disponível em: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html. Acesso em: 30 maio 2026.
- [7] RONNEBERGER, O.; FISCHER, P.; BROX, T. U-Net: Convolutional Networks for Biomedical Image Segmentation. In: INTERNATIONAL CONFERENCE ON MEDICAL IMAGE COMPUTING AND COMPUTER-ASSISTED INTERVENTION (MICCAI), 18., 2015, Munique. **Proceedings...** Cham: Springer, 2015. p. 234-241. Disponível em: <https://arxiv.org/abs/1505.04597>. Acesso em: 30 maio 2026.
- [8] PASZKE, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: NEURAL INFORMATION PROCESSING SYSTEMS (NEURIPS), 32., 2019, Vancouver. **Advances in Neural Information Processing Systems...** Red Hook: Curran Associates, 2019. p. 8024-8035. Disponível em: <https://arxiv.org/abs/1912.01703>. Acesso em: 30 maio 2026.

[9] PLOTLY. **Dash Documentation**: documentação oficial. [S. l.], 2026. Disponível em: <https://dash.plotly.com/>. Acesso em: 30 maio 2026.

[10] CONDA. **CEP 24 - Specification of environment.yml input files**: documentação oficial. [S. l.], 2026. Disponível em: <https://conda.org/learn/ceps/cep-0024/>. Acesso em: 30 maio 2026.

[11] MAMBA. **Mamba User Guide**: documentação oficial. [S. l.], 2026. Disponível em: https://mamba.readthedocs.io/en/stable/user_guide/mamba.html. Acesso em: 30 maio 2026.